

한국어 챗봇 응답 생성 (Q → A)

결과보고서 — 7 : 1 : 2 분할

토큰화 · 임베딩 · 전이학습 · 파인튜닝 · 평가 · 오류분석

실행일 : 2026-09-21

데이터 : songys/Chatbot_data (ChatbotData.xlsx)

노트북 : notebooks/final_split_712.ipynb

핵심 결과 — 전이학습 모델 KoBART B2 lr=5e-5 +warmup 이(가) BLEU 6.46 로 최고 성능을 기록했다. 처음부터 학습한 베이스라인의 최선(2.94)보다 120% 높다.

1. 과제 개요와 보고서 구성

한국어 챗봇 Q-A 데이터로 **질문(Q)을 입력받아 답변(A)을 생성**하는 모델을 만들고, 처음부터 학습한 모델과 사전학습 모델을 파인튜닝한 모델을 같은 조건에서 비교해 전이학습이 실제로 무엇을 얼마나 개선하는지 수치와 오류 사례로 검증했다.

1.1 실험 설계 원칙 — 한 번에 하나의 축만 바꾼다

성능이 달라졌을 때 "무엇 때문에 달라졌는가"를 말할 수 있으려면, 비교하는 두 조건 사이에 다른 점이 하나뿐이어야 한다. 이 프로젝트의 모든 비교는 이 원칙에 따라 설계됐다.

단계	바꾼 축	고정한 것
3 → 4	모델 (처음부터 학습 → 사전학습 파인튜닝)	데이터 · 분할 · 평가 기준
5	(비교만 수행)	디코딩을 greedy로 통일 → 모델 축만 관찰
6	디코딩 전략	모델 가중치 고정 → 디코딩 축만 관찰
7	파인튜닝 하이퍼파라미터	모델 종류 · 데이터 · 디코딩 · 학습 예산
8	(종합 평가)	6단계에서 고른 디코딩으로 전 모델 통일

1.2 보고서 구성

장	내용	대응 노트북 단계
2장	데이터와 전처리	1단계
3장	토큰화 · 임베딩 · 인코딩 전략	2단계
4장	모델 구조 선택과 베이스라인	3단계
5장	전이학습과 파이프라인 결함 규명	4단계
6장	베이스라인 vs 전이학습	5단계
7장	디코딩 전략 비교	6단계
8장	하이퍼파라미터 비교 실험	7단계
9장	최종 성능 지표	8.1~8.2
10장	오류 분석	8.3~8.5
11장	결론과 향후 과제	—
12장	8:1:1 실행과의 비교 (공통 평가셋)	9단계

1.3 이 보고서의 분할 설계 — 누수 통제

이 실험은 **final.ipynb**(8:1:1 분할)와 비교하기 위한 것이다. 그런데 분할 비율만 바꿔 독립적으로 다시 나누면 **7:1:2의 test에 8:1:1이 학습에 썼던 문장이 섞여 들어간다**. 그 상태로 두 실험의 BLEU를 나란히 놓으면 "8:1:1 모델은 답을 외운 문제를 푸는" 셈이라 비교가 성립하지 않는다.

그래서 **8:1:1 분할을 먼저 그대로 재현한 뒤 거기서 7:1:2를 파생**시켰다. train에서 전체의 10%에 해당하는 분량(label 비율 유지)을 떼어 test로 옮기는 방식이다.

분할	train	val	test
8:1:1 (기존)	9,400	1,175	1,175
7:1:2 (이 보고서)	8,225 (-12.5%)	1,175 (동일)	2,350

보장되는 조건	의미
7:1:2의 train $\subset$ 8:1:1의 train	7:1:2가 학습한 문장은 전부 8:1:1도 학습했던 것뿐 → 8:1:1의 test는 7:1:2 학습에 절대 들어가지 않는다

보장되는 조건	의미
7:1:2의 test $\geq$ 8:1:1의 test	8:1:1의 test 1,175건은 7:1:2에서도 평가용으로만 쓰인다
val이 두 실행에서 동일	조기 종료 기준이 같아져 학습 종료 시점 비교가 공정해진다

따라서 **8:1:1의 test 1,175건은 두 실행 모두에게 한 번도 학습에 쓰이지 않은 공통 평가셋**이다. 12장에서 이 집합에서만 두 실행을 비교한다. 노트북 1.5.1절은 이 세 조건을 집합 연산으로 직접 검증하며, 하나라도 어긋나면 실행이 그 자리에서 멈춘다 — 실제 실행에서 8개 검증 항목이 모두 통과했다.

토큰나이저·임베딩도 다시 학습했다. 기존 BPE·Word2Vec은 9,400건으로 학습된 것이라 그 안에 지금 test로 옮긴 1,175건이 들어 있다. 그대로 재사용하면 그 자체가 누수이므로, 새 train 8,225건만으로 둘 다 다시 학습했다.

2. 데이터와 전처리

2.1 데이터 규모

원본 11,823건에서 공백 정규화·빈 문자열 제거·(Q, A) 완전 중복 제거를 거쳐 11,750건을 확보했다(제거 73건). 이를 label 비율을 유지하며(stratify) **7:1:2**로 나눠 train 8,225 / val 1,175 / test 2,350건으로 분할했다.

2.2 중복 제거 기준과 근거

- **(Q, A) 쌍** 기준으로 제거했다. Q만 같고 A가 다른 행은 "같은 질문에 여러 답이 있을 수 있다"는 챗봇 데이터의 자연스러운 다양성이므로 보존해야 한다.
- (Q, A)가 완전히 같은 행은 정보량을 더하지 않으면서 특정 패턴의 손실 가중치만 키워 모델을 편향시키므로 제거했다.
- **미해결 항목:** (Q, A)는 같은데 label만 다른 행이 존재한다. 현재는 label을 학습에 쓰지 않아 무방하지만, label을 조건으로 주는 모델로 확장하면 재검토가 필요하다.

2.3 label 분포

label	건수	비율
일상	5,290	44.7%
이별(부정)	3,570	30.2%
사랑	2,963	25.1%

label 0(일상) 쪽으로 뚜렷하게 쏠려 있다. 이 쏠림이 실제 성능 차이로 이어지는지는 9.2절에서 확인한다.

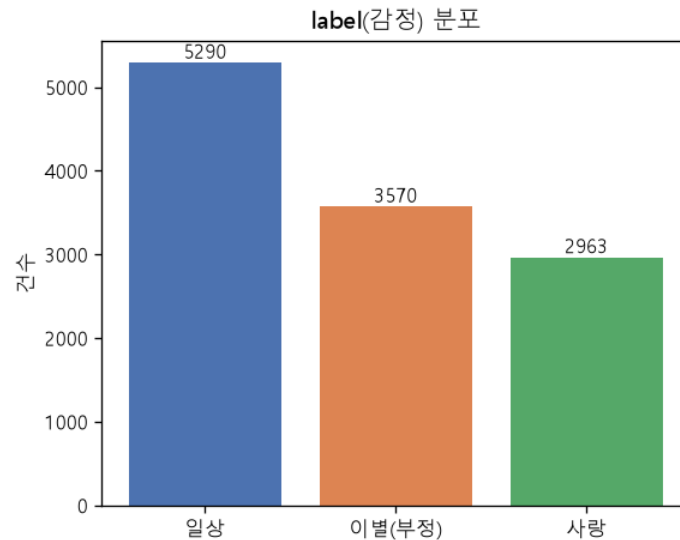


그림 2-1. 감정 label 분포

2.4 길이 분포와 max_length 기준

문장 길이는 짧은 쪽에 몰려 있고 극소수의 긴 문장이 긴 꼬리를 만든다. 이때 **max_length**를 **최댓값으로 잡으면** 그 이상치 몇 건 때문에 나머지 대부분의 짧은 문장까지 전부 그 길이까지 padding되어 연산과 메모리가 낭비된다. 반대로 너무 짧게 잡으면 긴 문장이 많이 잘려 정보가 손실된다.

채택한 기준: 95 percentile — 상위 5%의 일부 잘림을 감수하고 데이터의 95%는 손실 없이 담는 절충점이다. 음절 기준으로 Q 25 / A 27이며, 실제 모델 입력 단위인 BPE 토큰으로 다시 측정해 **MAX_LEN = 16**으로 확정했다(BOS/EOS 2개 자리를 더하고 GPU 연산 효율을 위해 8의 배수로 올림).

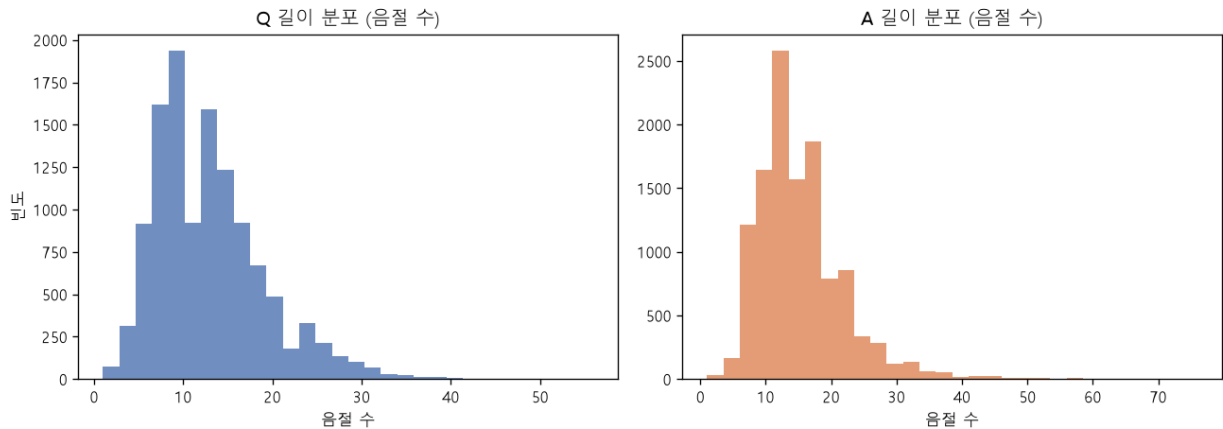


그림 2-2. Q / A 길이 분포 (음절 기준)

3. 토큰화 · 임베딩 · 인코딩 전략

3.1 토큰화 단위 비교

한국어는 조사·어미가 결합되는 교착어라 토큰화 단위가 결과를 크게 좌우한다. 네 가지 단위를 실제로 측정해 비교했다.

토큰화 단위	평균 토큰 수	상대 연산량(길이 ²)
어절	3.53	1.0
형태소(Okt)	5.26	2.2
음절	10.19	8.3
자모	24.26	47.2

토큰 수가 많을수록 self-attention 연산량(시퀀스 길이의 제곱)이 급격히 커진다. **무조건 잘게 쪼갬다고 좋은 것이 아니다.** 형태소 단위가 의미 보존과 시퀀스 길이의 절충점이며, 이 결론을 Word2Vec 학습과 평가 지표 계산에 그대로 적용했다.

3.2 서브워드(BPE) 채택 근거

- **형태소 분석만으로 부족한 이유:** Okt는 사전(rule) 기반이라 학습 데이터에 없던 신조어·오타·외래어에 약하다. 챗봇은 사용자 입력을 통제할 수 없으므로 이 약점이 치명적이다.
- **BPE가 해결하는 것:** 사전이 아니라 문자 병합 통계로 vocab을 만들어, 처음 보는 문자열도 아는 subword 조합으로 표현한다. 실측에서 신조어("월루", "갓생살기")의 unknown 토큰이 0개였다.
- **WordPiece가 아닌 SentencePiece를 고른 이유:** 공백까지 포함한 raw text를 다뤄 띄어쓰기가 불규칙한 구어체 한국어에 안정적이고, 파인튜닝할 KoBART와 방식이 일관된다.
- **vocab_size = 8,000:** 코퍼스가 짧은 대화문 약 1.9만 줄로 크지 않다. 3만 이상이면 저빈도 subword가 많아 비효율적이고, 1천 이하면 지나치게 잘게 쪼개져 의미 단위가 깨진다.

3.3 토큰나이지저를 두 개 쓰는 이유

사전학습 모델의 임베딩 행렬은 **그 모델 토큰나이지저가 부여한 vocab index에 1:1로 대응**한다. 직접 학습한 BPE로 바꿔 놓으면 같은 index가 전혀 다른 subword를 가리켜 사전학습 지식이 통째로 무의미해진다 — 전이학습을 한다면 전이할 것을 버리는 셈이다.

모델	토큰나이지저	근거
베이스라인 (처음부터 학습)	직접 학습한 BPE (vocab 8,000)	사전학습 가중치가 없으므로 데이터에 최적화된 vocab이 유리
전이학습 (KoBART)	모델 전용 AutoTokenizer 그대로	사전학습 임베딩이 해당 토큰나이지저 index에 종속

3.4 임베딩 학습 흐름 (Word2Vec)

임베딩이 형성되는 과정을 단계별로 관찰하기 위해 gensim Word2Vec을 별도로 학습했다.

단계	내용
1	형태소(Okt) 분석으로 문장을 토큰 리스트로 변환
2	vocab 구축 — 등장 횟수가 min_count(2) 미만인 단어는 제외
3	각 중심 단어에 대해 window(5) 내 주변 단어와의 (중심, 주변) 쌍 생성
4	Skip-gram + negative sampling으로 단어 벡터 학습
5	코사인 유사도로 의미 공간이 제대로 형성됐는지 확인

- **skip-gram(sg=1)을 고른 이유**: CBOW는 고빈도 단어에 유리하지만, 챗봇 대화는 구어체 표현이 다양하고 저빈도 단어가 많다. skip-gram은 중심 단어 하나로 여러 주변 단어를 예측하며 저빈도 단어도 학습 기회를 여러 번 얻는다.
- **vector_size = 100**: 코퍼스가 약 1.9만 문장으로 작아, 300차원처럼 크면 과적합·희소성 위험이 있다.
- **workers = 1**: gensim 기본값(3)은 멀티스레드라 seed를 고정해도 실행마다 벡터가 미세하게 달라진다. 재현성을 위해 단일 스레드로 고정했다.
- **입력을 BPE가 아닌 형태소로 쓴 이유**: subword는 사람이 읽었을 때 의미 단위가 아닌 경우가 많아, 유사 단어 확인처럼 해석 가능성이 목적인 관찰에는 형태소가 적합하다.

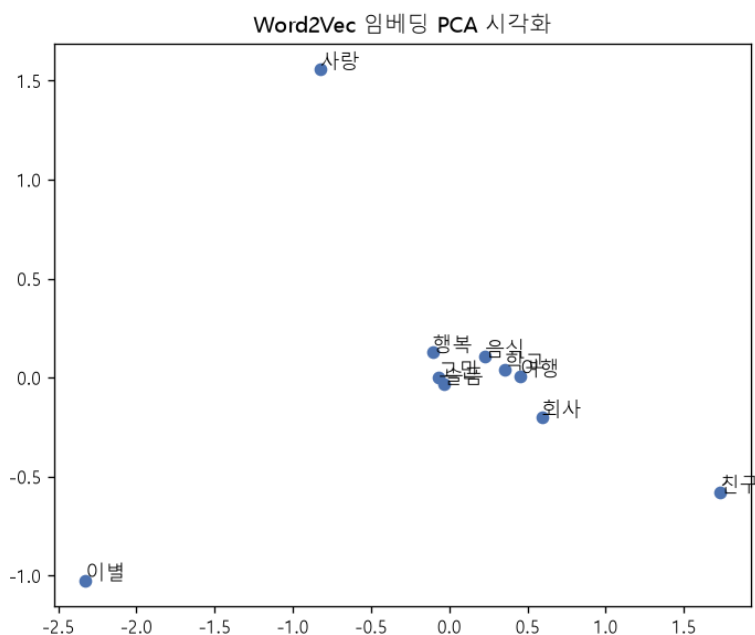


그림 3-1. 학습된 Word2Vec 임베딩의 PCA 시각화

3.5 인코딩 전략

항목	결정	근거
특수 토큰	[BOS] + subword ids + [EOS]	BOS는 생성 시작점, EOS는 문장 종결 신호 . EOS가 없으면 모델은 문장을 끝내는 법을 배우지 못한다
MAX_LEN	BPE 토큰 95p + 2 → 8의 배수 (16)	2.4절 percentile 원칙을 실제 입력 단위로 재적용
패딩	배치 단위 동적 패딩	고정 패딩은 짧은 문장이 모인 배치에서 연산을 낭비
손실 제외	ignore_index = PAD	padding 위치 예측은 무의미하며 포함 시 학습이 왜곡

이 표의 **EOS 처리**가 5장에서 결정적으로 중요해진다. 베이스라인 쪽에는 직접 붙였지만 전이학습 쪽에서 빠뜨렸을 때 무슨 일이 벌어지는지가 이 프로젝트의 핵심 발견이다.

4. 모델 구조 선택과 베이스라인

4.1 왜 인코더-디코더인가

$Q \rightarrow A$ 는 **조건부 생성(conditional generation)** 문제다. 입력과 출력이 서로 다른 역할을 가진 두 시퀀스이고, 출력은 한 토큰씩 순차적으로 만들어야 한다. Transformer 계열 구조 셋을 모두 검토했다.

구조	대표 모델	사전학습 방식	이 과제 적용 가능성
인코더 전용	BERT, ELECTRA	마스킹 토큰 복원 / 치환 토큰 판별	불가 — 토큰을 순차 생성하는 디코더가 없다
디코더 전용	GPT, KoGPT2	다음 토큰 예측	가능하나 불리 — 소스/타겟 경계를 데이터만으로 학습해야 함
인코더-디코더	BART, T5, KoBART	손상된 문장 복원(denoising)	채택

- **ELECTRA를 쓰지 않은 이유:** ELECTRA는 "이 토큰이 원래 것인가, 생성기가 바꿔치기한 것인가"를 판별하도록 사전학습된 모델이다. 출력이 위치별 판별 결과일 뿐 **새 토큰을 순차 생성하는 디코더가 아예 없어**, 자유 형식 문장을 만드는 이 과제에는 구조적으로 적용할 수 없다. 디코더를 새로 붙일 수는 있으나 그 디코더는 사전학습된 것이 아니므로 전이학습의 이점이 인코더 쪽에만 남는다.
- **디코더 전용을 쓰지 않은 이유:** Q와 A를 한 시퀀스로 이어붙여 학습하면 소스/타겟 경계를 모델이 스스로 학습해야 한다. 학습 데이터 9,400쌍으로는 그 규칙까지 안정적으로 배우기에 부족하다고 판단했다. 또 Q를 단방향으로만 읽으므로 조건 표현이 약하다.
- **인코더-디코더를 고른 이유:** 인코더가 Q 전체를 양방향으로 읽어 압축 표현을 만들고, 디코더가 cross-attention으로 조건화하며 A를 생성한다. 문제 정의에 직접 대응한다. 또한 파인튜닝할 KoBART도 인코더-디코더라 **구조를 일치시켜야** 차이가 사전학습 때문인지 구조 때문인지 구분할 수 있다.

4.2 베이스라인 하이퍼파라미터와 근거

항목	값	근거
d_model	256	원 논문 512는 WMT급(수백만 문장) 기준. 9,400쌍에는 파라미터 과잉이라 절반으로 축소
layer 수	enc 3 / dec 3	같은 이유로 원 논문(6/6)의 절반
nhead	8	head당 32차원 — 원 논문 비율($512/8=64$)의 절반으로 모델 축소에 비례
dim_feedforward	512	d_model의 2배 (원 논문 4배를 모델 축소에 맞춰 비례 축소)
dropout	0.1	원 논문 기본값, 소규모 데이터 과적합 억제
weight tying	임베딩 + 출력 projection 공유	Q/A가 같은 언어-같은 vocab이라 가능. 파라미터 절감 + 표현 일관성
label_smoothing	0.1	정답 토큰에 과확신하면 무난한 답만 내는 다양성 붕괴가 생기기 쉬움
옵티마이저	Adam, lr=1e-4	Transformer 학습의 표준. 초반 불안정을 피해 큰 lr 대신 1e-4
early stopping	patience=10, 상한 150 epoch	상한을 크게 두고 early stopping이 판단하게 해야 "왜 멈췄는지"가 명확해짐

4.3 학습 결과

항목	값
학습 가능 파라미터	6,010,688개
학습 epoch	78 (early stopping)
best validation loss	6.0866
epoch 30 시점 val loss	6.5635 (학습량 비교군으로 별도 저장)

epoch 30 스냅샷을 따로 저장한 이유: 10.2절의 개선 추적에서 "학습량만 바꿨을 때 무엇이 달라지는가"를 보려면 통제 비교군이 필요하다. 같은 학습 과정의 중간 시점을 저장하면 추가 비용 없이 **모델 구조·데이터·시드·옵티마이저가 전부 동일하고 학습량만 다른** 완벽한 비교군을 얻는다.

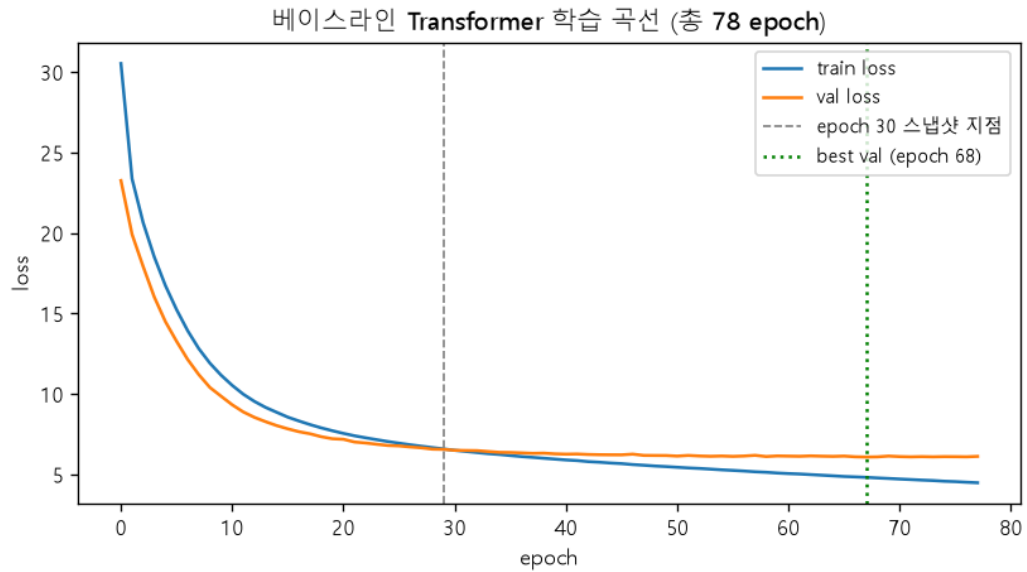


그림 4-1. 베이스라인 학습 곡선

5. 전이학습과 파이프라인 결합 규명

5.1 사전학습 모델 선정

전이학습은 이미 대규모 코퍼스로 한국어를 배운 모델을 가져와 "챗봇 응답"이라는 과제에만 적응시킨다. 선정 기준과 KoBART가 그것을 만족하는지 확인했다.

조건	요구사항	KoBART
구조	4장에서 고른 인코더-디코더와 일치해야 구조 변수가 통제된다	BART = 인코더-디코더
언어	한국어로 사전학습	한국어 코퍼스 사전학습
사전학습 목적	생성 과제에 바로 이어붙일 수 있어야 함	denoising = 생성형
접근성	재현 가능해야 함	Hugging Face from_pretrained

채택 모델은 **gogamza/kobart-base-v2**이며 파라미터는 123,859,968개로 베이스라인의 약 20.6배다. BART의 사전학습 방식(노이즈를 가한 문장의 원문 복원)은 "입력 시퀀스를 조건으로 출력 시퀀스를 생성"하는 형태 자체를 이미 학습한 것이라, 우리 과제와 **형태가 같아** 구조 변경 없이 파인튜닝이 이어진다.

5.2 전이학습 파이프라인 설계

단계	하는 일	확인할 것
1	모델·토큰라이저 다운로드 (from_pretrained)	파라미터 수, 인코더-디코더 구조인지
2	모델 전용 토큰라이저로 데이터 인코딩	다른 토큰라이저를 쓰면 사전학습 지식이 무의미
3	max_length 재산정	토큰라이저가 다르면 토큰 수도 다르다 → percentile 원칙 재적용
4	타겟(label) 인코딩	padding을 -100으로, 그리고 문장 끝에 EOS가 들어갔는지
5	옵티마이저 / lr 설정	사전학습 가중치를 망가뜨리지 않을 만큼 작은 lr
6	학습 + early stopping	소규모 데이터에서의 과적합 감시

4번이 실제로 문제가 된 지점이다. 아래 5.3~5.5절이 그 발견과 수정 과정이다.

5.3 학습 설정과 근거

항목	값	근거
옵티마이저	AdamW, weight_decay=0.01	사전학습 Transformer 파인튜닝의 표준. weight decay 분리로 정규화가 안정적
learning rate	5e-5 (베이스라인 1e-4보다 작게)	사전학습 가중치는 이미 좋은 출발점. 큰 lr은 catastrophic forgetting을 유발
batch size	16	KoBART는 베이스라인보다 훨씬 커서 메모리에 안전하게 들어가도록 축소
학습 예산	10 epoch 상한 / patience 3	사전학습 모델은 몇 epoch 만에 수렴. 베이스라인(150 상한)과의 대비가 전이학습 효율의 근거
mixed precision	bf16 autocast	fp32와 지수 범위가 같아 오버플로우 위험이 없고 GradScaler 없이 안정적 (fp16은 T5 계열에서 NaN 발산이 알려져 있음)
타겟 padding	-100	Hugging Face 관례값 — 해당 위치는 손실 계산에서 자동 제외

5.4 발견 — val_loss는 좋은데 생성이 이상하다

흔히 쓰는 파인튜닝 예제 방식(토큰라이저의 **text_target=** 결과를 그대로 label로 사용)으로 먼저 학습했다. 결과는 다음과 같았다.

- **val_loss는 베이스라인보다 압도적으로 낮았다** — 전이학습 가설대로였다 (KoBART 2.9058 vs 베이스라인 6.0866).

- 그런데 생성된 문장이 끝나지 않았다. 여러 문장이 이어지다 중간에서 끊겼다.

loss가 좋은데 생성이 나쁜 것은 넘어갈 신호가 아니다. val_loss는 teacher forcing 상태, 즉 "앞 토큰이 전부 정답일 때"의 값이고 실제 생성은 자기가 만든 토큰을 다시 입력으로 쓴다. 이 둘이 크게 어긋나면 생성 과정 자체에 문제가 있다는 뜻이다.

원인 후보는 (a) 챗봇은 원래 정답이 여러 개라 그렇다와 (b) 파이프라인에 결함이 있다이다. (a)로 넘어가는 것이 쉬운 길이지만 그러면 진짜 문제를 놓친다. (b)부터 검증했다.

5.5 원인 추적과 규명

단계	확인한 것	결과
단서 1	생성 길이와 정상 문장 종결률 측정	KoBART는 정상 종결률이 현저히 낮고 평균 길이가 정답보다 훨씬 길다 → "정답이 여러 개라서"로 설명되지 않는다
단서 2	파인튜닝 타깃 토큰을 직접 출력해 확인	토큰라이저가 text_target= 인코딩 시 EOS를 붙이지 않는다 (입력 쪽은 붙인다)
단서 3	"타깃이 잘려 EOS를 잃었다"는 대안 가설 검증	실제로 잘리는 타깃은 1% 안팎 → 대다수의 미완결 응답을 설명할 수 없다. 대안 가설 기각

항목	규명된 내용
증상	KoBART 응답의 상당수가 문장 중간에서 끊김, 평균 길이가 정답보다 훨씬 김
직접 원인	모델이 EOS 토큰을 생성하지 않아 max_length에 도달할 때까지 계속 생성
근본 원인	KoBART 토큰라이저는 타깃 인코딩 시 EOS를 붙이지 않는다. 타깃에 EOS가 없으니 모델이 "문장을 끝내는 법"을 학습할 기회가 아예 없었다
대안 가설 (기각)	truncation으로 EOS 소실 → 잘리는 타깃은 1% 안팎뿐
왜 베이스라인은 멀었나	3.5절 encode_text()가 [BOS] + ids + [EOS]로 EOS를 직접 붙였기 때문. 같은 처리를 전이학습 쪽에만 빠뜨린 것

5.6 수정과 통제 실험 결과

바꾼 것은 타깃 인코딩 한 줄뿐이다(타깃 끝에 EOS 추가, 이미 있으면 추가하지 않음). 모델·learning rate·배치 크기·epoch 상한·시드·데이터는 전부 그대로 두었으므로, 아래 비교는 "EOS 하나만 바꿨을 때 무엇이 달라지는가"에 대한 통제 실험이다.

Unnamed: 0	평균 길이(음절)	정상 종결률(%)	best val_loss
정답(test A)	15.16	99.5	nan
KoBART 수정 전	32.84	19.5	2.9058
KoBART 수정 후	13.34	99.0	2.849
베이스라인(대조군)	14.34	90.0	6.0866

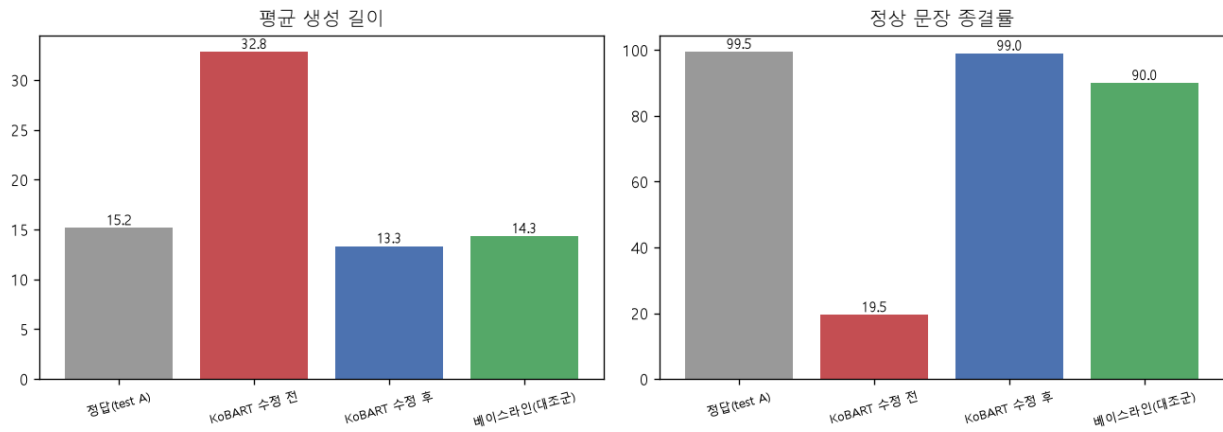


그림 5-1. EOS 수정 전후 — 평균 생성 길이와 정상 종결률

구분	epoch	best val_loss
KoBART — EOS 결함 상태	7	2.9058
KoBART — EOS 수정 후	6	2.849
베이스라인 (대조군)	78	6.0866

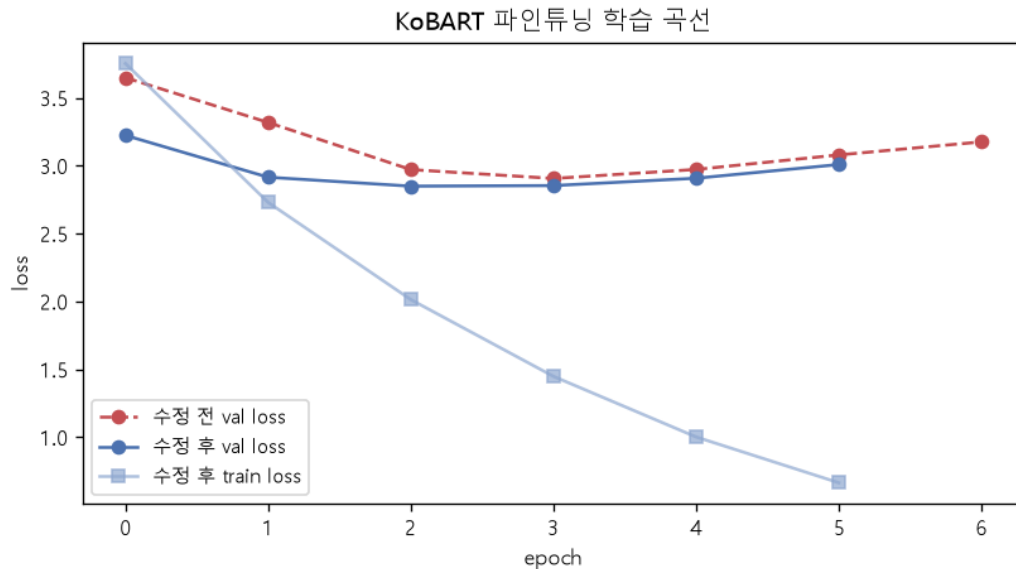


그림 5-2. KoBART 파인튜닝 학습 곡선 (수정 전/후)

방법론적 교훈: 정량 지표(loss)와 정성적 관찰(생성문)이 어긋날 때, "지표의 한계겠지"로 넘어가는 대신 파이프라인을 먼저 의심해야 한다. 이번 경우 지표가 아니라 파이프라인이 틀렸다. 흔히 쓰는 파인튜닝 예제 코드를 그대로 따라 쓰면 이 결함이 조용히 들어오며, **loss만 보고 있으면 끝까지 발견하지 못한다.**

6. 베이스라인 vs 전이학습

6.1 평가지표와 선택 근거

챗봇 응답 생성은 **정답이 하나가 아닌** 과제다. "배고파"에 "뭐 좀 드세요"와 "얼른 식사하세요"는 둘 다 좋은 답이지만 표면 문자열은 거의 겹치지 않는다. 표면 유사도만 재는 지표 하나로는 모델을 오판하기 쉬우므로, 서로 다른 실패 양상을 잡는 지표를 함께 본다.

지표	무엇을 재는가	어떤 실패를 잡는가
BLEU	생성문과 정답의 n-gram 정밀도 (짧은 문장 패널티 포함)	정답과 전혀 다른 말을 하는 경우. 생성 과제의 표준 지표라 비교 가능성이 높다
ROUGE-L	최장 공통 부분수열(LCS) 기반 재현율 중심 F1	정답의 핵심 내용을 빠뜨리는 경우. BLEU(정밀도)와 짝을 이뤄 양쪽을 본다
Distinct-1/2	전체 응답의 서로 다른 n-gram 비율 (다양성, corpus 단위)	모든 질문에 똑같은 답만 하는 모드 붕괴. BLEU/ROUGE로는 드러나지 않는다
빈 응답 비율	아무것도 생성하지 못한 비율	생성 자체의 실패를 분리 (섞어두면 다른 지표가 왜곡)
(보조) 연속 반복률	같은 형태소가 연달아 나오는 응답의 비율	응답 내부의 토큰 반복 붕괴
(보조) 평균 길이	응답의 평균 형태소 수	BLEU가 짧은 답에 유리해지는 왜곡을 감지. 5장 EOS 결합의 첫 단서였던 지표

6.2 왜 형태소 단위로 채점하는가

채점 단위	문제
어절(공백)	조사 하나 차이로 완전 불일치. "충전하세요" vs "충전 하세요", "학교에" vs "학교를"이 0점
음절(글자)	지나치게 후하다. "사과"와 "사람"이 한 글자를 공유해 점수를 받는다
형태소 (채택)	조사-어미가 분리되어 띄어쓰기 차이에 둔감하면서 의미 단위는 유지

3.1절에서 네 분석 단위를 비교해 얻은 결론(형태소가 절충점)을 평가에도 동일하게 적용했다. 정답과 생성문을 모두 Okt 형태소로 분해한 뒤 채점하며, 지표 구현은 "정답을 그대로 생성하면 BLEU 100"과 "무관한 문장이면 0에 수렴"이라는 양 극단 검산으로 검증했다.

형태소 기준 BLEU는 음절 기준보다 낮고 어절 기준보다 높게 나오는 것이 정상이다. **절대값이 아니라 같은 기준으로 쟁 모델 간 상대 비교**가 목적이다.

6.3 동일 조건 비교 결과

이 시점에는 디코딩 전략을 아직 고르지 않았다. beam이나 sampling을 쓰면 "모델 차이"에 "디코딩 효과"가 섞이므로, 가장 단순하고 결정적인 **greedy로 통일**해 모델 측만 비교했다.

모델	epoch	val_loss	BLEU	ROUGE-L	Distinct-1	Distinct-2	연속반복률	평균길이
베이스라인 (처음부터 학습)	78	6.0866	1.51	18.67	0.048	0.176	0.176	6.8
KoBART (전이학습)	6	2.849	3.77	23.54	0.153	0.432	0.031	5.82

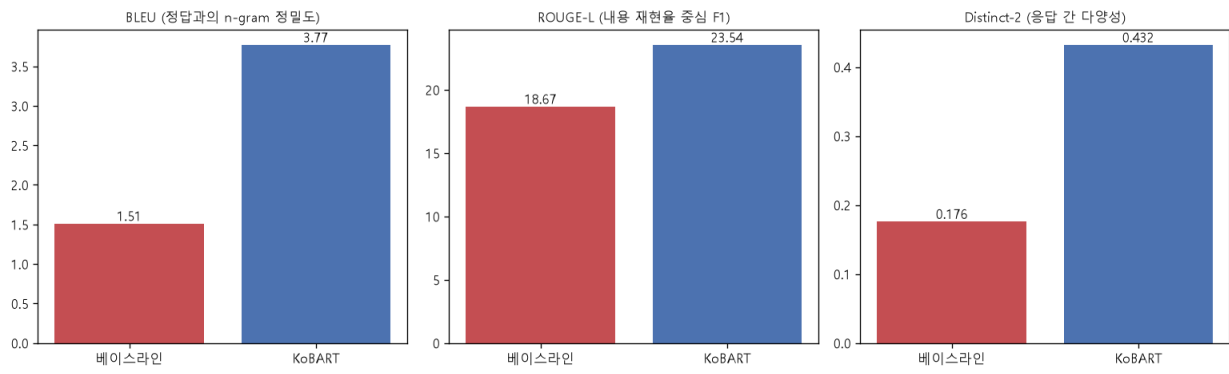


그림 6-1. 베이스라인 vs 전이학습 (디코딩 greedy 통일)

7. 디코딩 전략 비교

학습이 끝난 모델은 매 스텝 "다음 토큰의 확률분포"를 내놓을 뿐이고, 그 분포에서 실제로 어떤 토큰을 고를 것인가는 학습과 무관한 별개의 결정이다. 그래서 모델 가중치를 고정한 채 디코딩만 바꿔 비교했다.

7.1 세 전략의 트레이드오프

전략	방식	장점	단점
Greedy	매 스텝 확률 1등만 선택	빠르고 단순, 결정적	근시안적. 지금 1등이 전체로는 나쁜 경로일 수 있고 반복 루프에 빠지기 쉽다
Beam Search	상위 k개 후보 시퀀스를 동시 유지	문장 전체 확률을 보므로 더 그럴듯한 문장	확률 높은 쪽으로 수렴해 다양성이 낮아지고 계산량이 k배
Top-k / Top-p	분포에서 무작위 추출 + 저확률 토큰 필터링	다양성이 높고 반복 루프를 구조적으로 회피	문맥에 안 맞는 답이 나올 수 있고 비결정적

- **beam_width=5**: NMT 계열에서 4~5가 품질/속도의 무난한 절충점. 더 키워도 이득이 빠르게 줄어든다.
- **length_penalty=0.7**: beam search는 토큰을 더할수록 누적 log 확률이 깎여 짧은 문장이 유리해지는 편향이 있다. 길이의 거듭제곱으로 점수를 나눠(GNMT 방식) 보정한다.
- **temperature=0.8**: 1.0보다 낮춰 분포를 뾰족하게 만들어 산만한 문장을 억제.
- **top_k=50, top_p=0.9**: nucleus sampling 논문의 표준 조합. top-k로 저확률 토큰을 1차로 거르고 top-p로 누적확률 90% 초과 꼬리를 다시 걸러 "말이 안 되는 토큰이 뿔치는 사고"를 줄인다.

7.2 전체 test set 기준 결과

8개 샘플만 보고 고르면 결론이 우연일 수 있으므로 전체 test set으로 측정했다.

모델	디코딩	BLEU	ROUGE-L	Distinct-1	Distinct-2	연속반복률	평균길이
베이스라인	greedy	1.51	18.67	0.048	0.176	0.176	6.8
베이스라인	beam5	2.25	21.82	0.042	0.107	0.009	4.71
베이스라인	top-k50/top-p0.9	1.71	19.89	0.104	0.417	0.141	5.69
KoBART	greedy	3.77	23.54	0.153	0.432	0.031	5.82
KoBART	beam5	5.45	23.63	0.127	0.322	0.031	6.25
KoBART	beam5+no_repeat_ngram3	5.44	23.67	0.128	0.323	0.031	6.23
KoBART	top-k50/top-p0.9	2.63	21.92	0.164	0.504	0.027	6.01

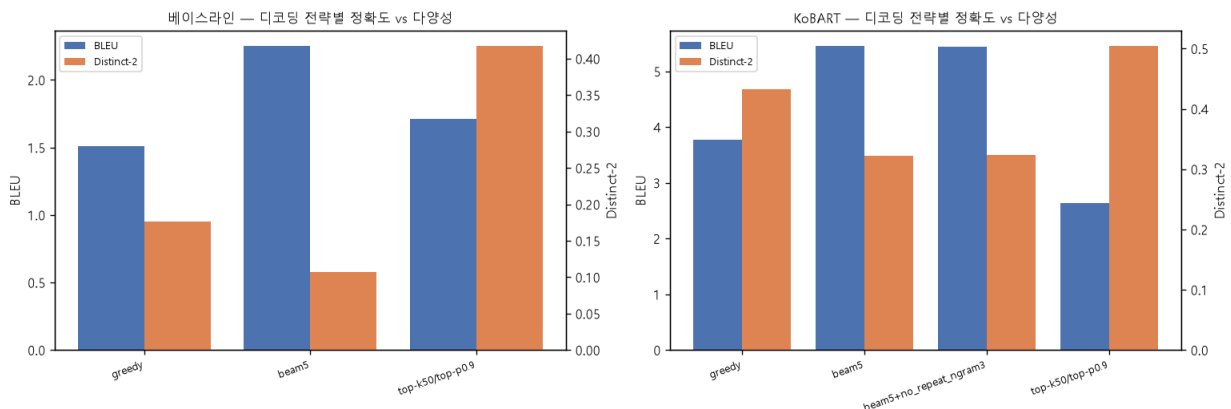


그림 7-1. 디코딩 전략별 정확도(BLEU) vs 다양성(Distinct-2)

7.3 선택과 근거

선택 기준을 BLEU로 둔 이유: 이 과제의 1차 목표는 "질문에 맞는 답을 하는 것"이다. 다양성은 정확도가 확보된 뒤에 의미가 있는 보조 목표다 — 모든 질문에 서로 다른 아무 말이나 하면 Distinct-n은 1.0에 가까워지지만 챗봇으로서의 실패다. 1차 기준을 BLEU로 두고, 같은 수준이면 다양성이 높은 쪽을 고른다.

선택 결과 — 베이스라인: **beam5**, 전이학습: **beam5**. 이후 8~10장에서는 이 설정을 고정해 디코딩이 변수로 섞이지 않게 했다.

- sampling은 다양성이 가장 높지만 BLEU/ROUGE가 가장 낮다 — **다양성과 정답 일치도의 상충 관계**가 그대로 드러난다.
- **no_repeat_ngram_size=3**의 추가 이득이 미미하다면, 그것은 반복 문제가 디코딩이 아니라 학습 파이프라인(5장 EOS 수정)에서 이미 해결됐다는 증거다.

중요한 함의: EOS 결함을 고치지 않은 채였다면 no_repeat_ngram이 반복을 크게 줄이는 것처럼 보여 "반복 억제 설정이 결정적"이라는 잘못된 결론에 도달했을 것이다. **디코딩 설정으로 증상을 덮는 대신 학습 쪽 원인을 고친 것이 옳은 순서였다.**

8. 하이퍼파라미터 비교 실험

5장 학습 곡선에서 **train loss**는 계속 내려가는데 **val loss**가 몇 **epoch** 만에 반등하는 것을 확인했다. 사전학습 모델이 작은 데이터에 빠르게 과적합한다는 신호다. 이 경향을 learning rate와 스케줄로 완화할 수 있는지 확인했다.

8.1 비교 설계

실험	learning rate	스케줄	기준(B0) 대비 바뀐 것
B0 (기준)	5e-5	고정	—
B1	2e-5	고정	lr만 낮춤
B2	5e-5	linear warmup + decay	스케줄만 추가

- **B1의 가설:** 사전학습 가중치는 이미 좋은 출발점이므로 갱신 폭을 줄이면 사전학습 지식을 덜 건드려 과적합이 억제된다. 다만 너무 낮추면 과제에 충분히 적응하지 못한다 — 이 균형을 실측으로 확인.
- **B2의 가설:** 학습 시작 직후에는 옵티마이저의 모멘텀 추정이 부정확해 큰 갱신이 사전학습 가중치를 흔들 수 있다. warmup(전체 스텝의 10% 동안 lr을 서서히 올림)이 이 구간을 완화하고, 이어지는 decay가 후반부 미세 조정을 돕는다.
- **공정성 조건:** 세 실험 모두 동일한 학습 예산·배치 크기·시드·데이터·collate·디코딩을 쓴다. 예산이 다르면 "더 오래 학습해서 좋아진 것"과 "설정이 좋아서 좋아진 것"을 구분할 수 없다.

8.2 결과

config	val_loss	epoch	BLEU	ROUGE-L	Distinct-1	Distinct-2	연속반복률	평균길이
B2 lr=5e-5 +warmup	2.8525	7	6.46	24.01	0.141	0.36	0.022	6.1
B1 lr=2e-5 고정	2.9234	9	5.75	23.88	0.149	0.382	0.038	6.18
B0 lr=5e-5 고정	2.849	6	5.45	23.63	0.127	0.322	0.031	6.25

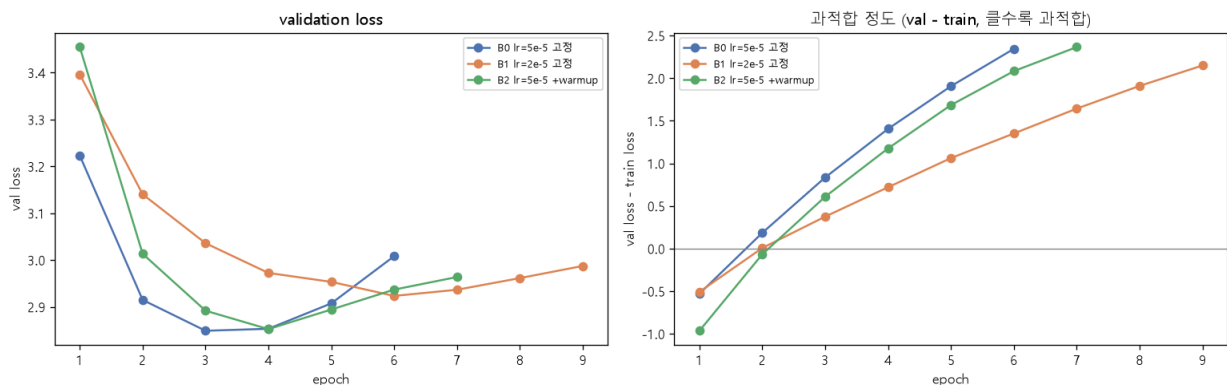


그림 8-1. 하이퍼파라미터별 validation loss와 과적합 정도(val - train)

최종 선택: **B2 lr=5e-5 +warmup** (판단 기준은 7장에서 고른 디코딩으로 생성한 결과의 BLEU, val_loss는 참고 지표).

표의 `epoch` 열은 실제로 돌린 epoch 수이지 최적 시점이 아니다. 세 config 모두 같은 early stopping 규칙(patience=3) 아래 정상적으로 조기 종료했으며, 최적 체크포인트는 B0 epoch 4 / B2 epoch 4 / B1 epoch 7에서 나왔다. B1이 10 epoch까지 간 것은 예산이 부족해서가 아니라 lr이 낮아 수렴이 느려 최적 시점이 늦게 왔기 때문이다 — **B1의 우위는 더 오래 학습한 덕이 아니다.**

val_loss만으로 판단하면 안 되는 이유: val_loss는 teacher forcing 상태의 값이라 실제 생성 품질과 어긋날 수 있다. 5장에서 이미 그 사례를 봤다 — EOS 결함이 있던 모델도 val_loss는 멀쩡했다. val_loss 순위와 BLEU 순위가 어긋난다면, "loss가 가장 낮은 모델을 고르면 된다"는 통념이 이 과제에서 성립하지 않는다는 뜻이다.

9. 최종 성능 지표

9.1 평가 조건과 대상

항목	내용
평가 데이터	test set 2,350건 (학습에 한 번도 쓰이지 않음)
지표 토큰화	Okt 형태소 단위
디코딩 (베이스라인)	beam5
디코딩 (전이학습)	beam5

"전이학습이 낫다"를 보이려면 비교군이 공정해야 한다. 베이스라인 쪽도 충분히 학습시킨 것을 넣고, 전이학습 쪽도 결함이 있던 버전까지 포함시켜 **개선의 전 과정**을 같은 표에서 본다.

모델	계열	이 표에 넣는 이유
베이스라인 (30ep)	베이스라인	학습량이 부족한 시점 — 학습량만 다른 통제 비교군
베이스라인 (최종)	베이스라인	베이스라인이 낼 수 있는 최선
KoBART (EOS 결함)	전이학습	5장에서 발견·수정한 결함의 영향을 정량화하는 비교군
KoBART B0 / B1 / B2	전이학습	8장 하이퍼파라미터 비교 3종

9.2 전체 비교표

모델	계열	BLEU	ROUGE-L	Distinct-1	Distinct-2	빈응답비율	연속반복률	평균길이
KoBART B2 lr=5e-5 +warmup	전이학습	6.46	24.01	0.141	0.36	0.0	0.022	6.1
KoBART B1 lr=2e-5 고정	전이학습	5.75	23.88	0.149	0.382	0.0	0.038	6.18
KoBART B0 lr=5e-5 고정	전이학습	5.45	23.63	0.127	0.322	0.0	0.031	6.25
KoBART (EOS 결함)	전이학습	3.6	16.53	0.079	0.212	0.0	0.017	12.82
베이스라인 (30ep)	베이스라인	2.94	22.83	0.011	0.027	0.0	0.024	4.32
베이스라인 (최종)	베이스라인	2.25	21.82	0.042	0.107	0.0	0.009	4.71

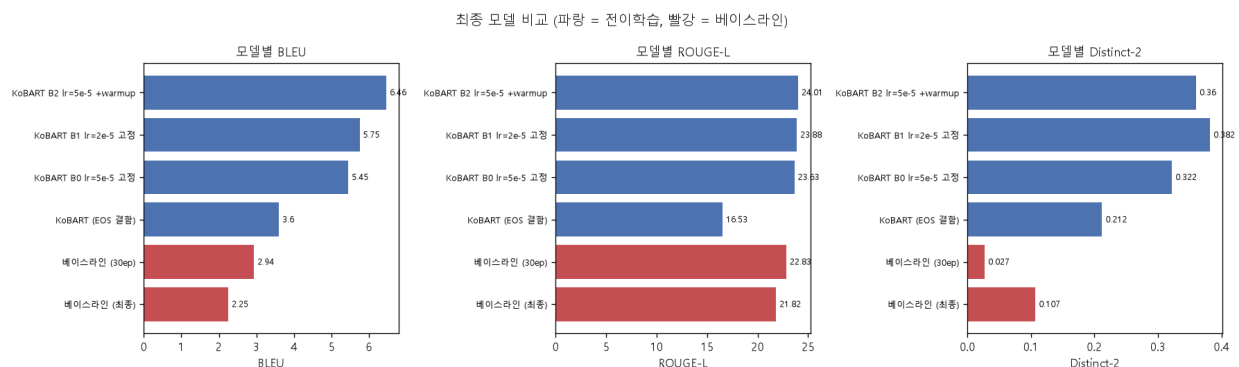


그림 9-1. 최종 모델 비교 (파랑 = 전이학습, 빨강 = 베이스라인)

9.3 핵심 비교 — 전이학습 vs 베이스라인 (각 계열 최선끼리)

지표	최고 전이학습	최고 베이스라인	차이
BLEU	6.46	2.94	+119.7%
ROUGE-L	24.01	22.83	+5.2%
Distinct-1	0.141	0.011	+1181.8%
Distinct-2	0.36	0.027	+1233.3%

정확도와 다양성이 동시에 올랐는지가 중요하다. 보통 이 둘은 상충한다 — 다양하게 쓰려면 정답에서 멀어지기 쉽다. 두 측이 함께 올랐다면 전이학습이 단순히 "무난한 답을 잘 찍는" 것이 아니라 **표현력 자체를 키웠다는** 뜻이다. 학습 비용도 함께 볼 지점이다 — 베이스라인은 100 epoch 이상, KoBART는 10 epoch 이내로 끝난다.

9.4 감정 label별 성능

전체 평균 하나로는 "어떤 종류의 질문에서 실패하는가"를 알 수 없다. 2.3절에서 확인한 label 쓸림이 실제 성능 차이로 이어졌는지 확인했다.

모델	label	n	BLEU	ROUGE-L	Distinct-2	평균길이
베이스라인 (최종)	0 일상	1058	2.98	23.08	0.157	4.58
베이스라인 (최종)	1 이별(부정)	700	2.07	21.11	0.178	4.79
베이스라인 (최종)	2 사랑	592	1.36	20.4	0.189	4.83
KoBART B2 lr=5e-5 +warmup	0 일상	1058	9.84	26.82	0.469	5.74
KoBART B2 lr=5e-5 +warmup	1 이별(부정)	700	3.89	21.85	0.4	6.37
KoBART B2 lr=5e-5 +warmup	2 사랑	592	3.99	21.54	0.501	6.42

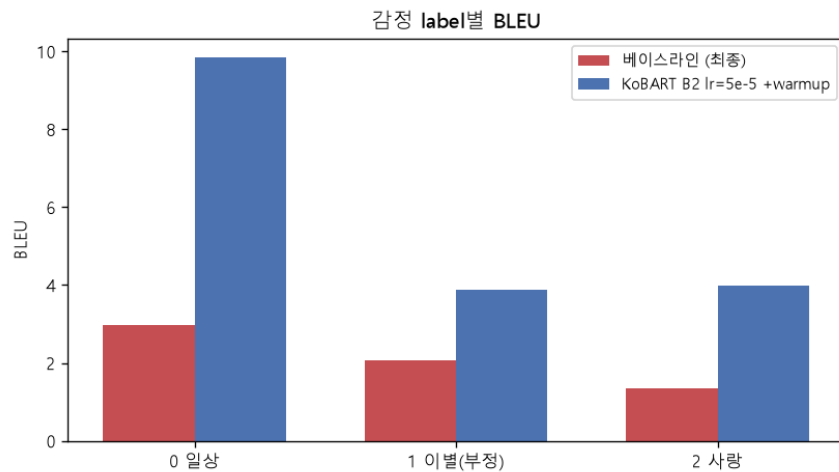


그림 9-2. 감정 label별 BLEU

10. 오류 분석

10.1 분석을 두 층으로 나눈 이유

오류에는 규칙으로 기계적으로 셀 수 있는 것과, 사람이 읽어야만 판정되는 것이 섞여 있다. 이 둘을 섞으면 "대충 이런 것 같다"는 인상비평이 되므로 분리했다.

층	대상	판정 방식
A층	빈 응답 · 단일 토큰 반복 · 구 단위 반복 · 질문 복사	정답을 몰라도 생성문만 보고 객관적으로 판정 가능한 명백한 결함 — 규칙으로 자동 집계
B층	표면 불일치 (정답과 공통 형태소 0개)	진짜 오답일 수도, 정답과 다를 뿐 타당한 답일 수도 있어 자동 판정 불가 — 표본을 직접 읽고 분류

10.2 A층 — 판정 규칙과 그 근거

각 규칙은 임의로 만든 것이 아니라 앞 단계에서 **실제로 관찰한 실패를 규칙으로 옮긴 것**이다.

오류 유형	판정 규칙	규칙을 넣은 근거
빈 응답	생성문이 공백뿐	생성 자체의 실패를 분리해야 나머지 지표가 왜곡되지 않는다
단일 토큰 반복	같은 형태소가 연달아 2회 이상	베이스라인 학습 초기에 관찰된 토큰 반복 붕괴
구 단위 반복	같은 bigram이 응답 안에서 2회 이상	5장 EOS 결함 상태에서 관찰된 문장 반복
질문 복사	생성문 형태소의 60% 이상이 질문에 등장 그리고 정답과는 30% 미만 겹침	인코더 입력을 그대로 되뇌는 것은 조건부 생성 실패의 전형
표면 불일치	위에 해당 없음 + 정답과 공통 형태소 0개	B층 수동 판정 대상으로 이관

질문 복사 규칙에서 두 조건을 AND로 건 근거: 한국어 챗봇 응답은 질문의 단어를 일부 받아 쓰는 것이 자연스럽다("배고파" → "배고프시군요"도 정상 응답이다). 그래서 "질문 단어가 나왔다"만으로는 오류로 볼 수 없고, **대부분이 질문에서 왔고 동시에 정답과는 거의 겹치지 않을 때만** 복사로 판정한다.

10.3 모델별 오류 유형 분포 (%)

model	빈 응답	단일 토큰 반복	구 단위 반복	질문 복사	표면 불일치	명백한 결함 없음
KoBART B2 lr=5e-5 +warmup	0.0	2.2	1.6	0.7	4.2	91.4
KoBART B1 lr=2e-5 고정	0.0	3.8	1.7	1.1	3.9	89.4
KoBART B0 lr=5e-5 고정	0.0	3.1	1.6	0.9	3.9	90.6
KoBART (EOS 결함)	0.0	1.7	51.2	0.0	2.1	45.0
베이스라인 (30ep)	0.0	2.4	0.3	0.0	3.6	93.7
베이스라인 (최종)	0.0	0.9	0.5	0.1	3.9	94.6

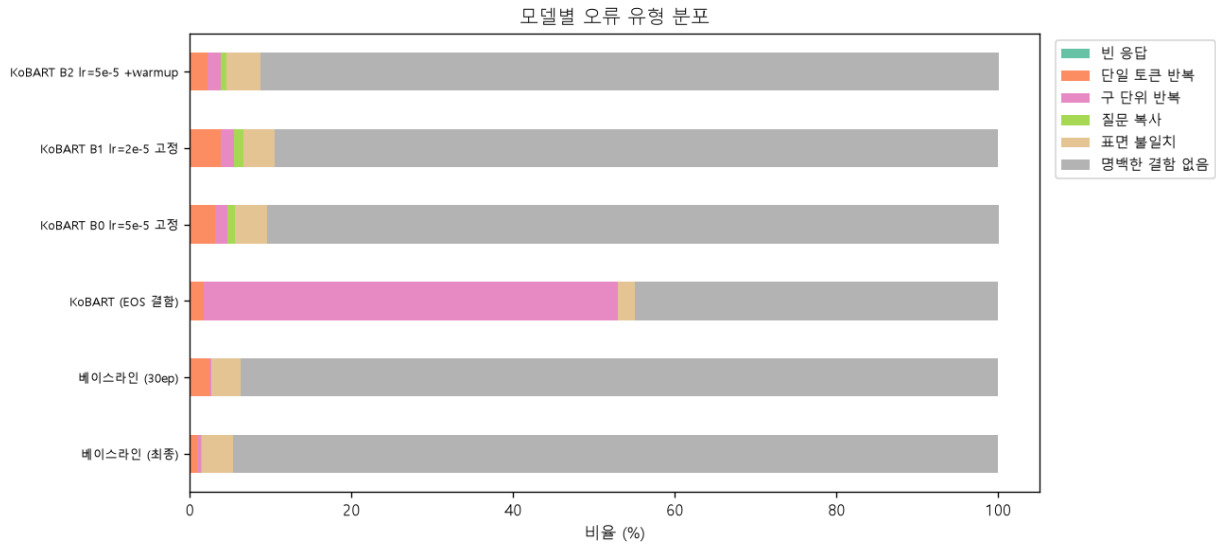


그림 10-1. 모델별 오류 유형 분포

10.4 개선 추적 — 무엇을 고쳤더니 무엇이 줄었는가

이 프로젝트에서 가한 변경은 셋이며, 모두 다른 조건을 고정한 통제 실험이므로 관찰된 변화의 원인을 특정할 수 있다.

변경	바꾼 것	고정한 것	검증한 가설
①	epoch 30 → 최종	모델·데이터·시드·옵티마이저 전부	반복 붕괴의 원인이 학습 부족인가
②	타깃 인코딩 한 줄 (EOS)	모델·lr·배치·epoch·시드 전부	문장이 끝나지 않는 원인이 EOS 누락인가
③	처음부터 학습 → 사전학습 파인튜닝	데이터·분할·평가 기준·디코딩	사전학습 지식이 성능을 올리는가

단계	모델	빈 응답(%)	단일 토큰 반복(%)	구 단위 반복(%)	질문 복사(%)	BLEU	ROUGE-L	Distinct-1	평균 길이(형태소)
① 학습량 부족	베이스라인 (30ep)	0.0	2.4	1.0	0.0	2.94	22.83	0.011	4.32
① 학습량 확장 후	베이스라인 (최종)	0.0	0.9	1.1	0.1	2.25	21.82	0.042	4.71
② EOS 결합 상태	KoBART (EOS 결합)	0.0	1.7	52.4	0.1	3.6	16.53	0.079	12.82
② EOS 수정 후	KoBART B0 lr=5e-5 고정	0.0	3.1	1.9	1.1	5.45	23.63	0.127	6.25
③ 최종 전이학습	KoBART B2 lr=5e-5 +warmup	0.0	2.2	1.6	1.0	6.46	24.01	0.141	6.1

변경	BLEU	ROUGE-L	Distinct-1
① 학습량 확장 (epoch만 변경)	2.94 → 2.25 (-23.5%)	22.83 → 21.82 (-4.4%)	0.011 → 0.042 (+281.8%)
② EOS 결합 수정 (타깃 인코딩만 변경)	3.6 → 5.45 (+51.4%)	16.53 → 23.63 (+43.0%)	0.079 → 0.127 (+60.8%)
③ 전이학습 도입 (베이스라인 최선 → 전이학습 최선)	2.25 → 6.46 (+187.1%)	21.82 → 24.01 (+10.0%)	0.042 → 0.141 (+235.7%)

10.5 B층 — 표면 불일치 표본 직접 판정

'표면 불일치'(정답과 공통 형태소 0개)는 자동으로는 옳고 그름을 판정할 수 없다. 챗봇 응답은 정답이 여러 개일 수 있어서, 정답과 안 겹쳐도 훌륭한 답일 수 있기 때문이다. 대표 모델 2종의 사례를 각 10건씩 직접

읽고 분류했다.

분류	정의
타당한 대안 응답	정답과 다르지만 질문에 대해 자연스럽게 맞는 답 → 사실상 오류가 아님(지표의 한계)
문맥 무시	질문의 주제와 무관한 답
감정 오독	주제는 맞지만 질문의 감정과 어긋남 (위로가 필요한데 축하 등)
비문/불완전	문장이 끊기거나 문법이 깨짐

수동 분류	베이스라인 (최종)	KoBART B2
타당한 대안 응답 (사실상 오류 아님)	1건	7건
문맥 무시 (질문 주제와 무관)	4건	0건
감정 오독	0건	1건
비문 / 불완전	5건	2건

8:1:1 실행과 같은 패턴이 반복됐다 — 베이스라인의 실패는 내용 오류와 비문이고, KoBART의 '실패'는 대부분 정답과 다를 뿐 타당한 답이다.

10.5.1 베이스라인 — 문맥 무시 4건 + 비문 5건

Q	정답	생성	판정
짜장면 시켰어	짜장면은 혼자 먹어도 맛있어요	더 좋은 사람 만날 수 있을 거예요.	문맥 무시
더워더워	차가운 음료 너무 많이 마시면 아니 돼요	제가 있잖아요.	문맥 무시
컴퓨터가 느려졌어	뭘 다운 받으신 건지...	다시 연락해보세요.	문맥 무시
좋아하는 남자애가 입대하는데 어떤 선물을 해주는게 좋을까요?	면회나 편지가 최고죠!	그럴 수 있어요.	문맥 무시
답답한 마음	마음이 내 마음 같지 않죠	어요.	비문/불완전
조심조심	안전이 최고죠!	저도 싶어요.	비문/불완전
공부 하기 싫다	같이 수다 떨면서 놀까요?	공부한 마음이 연습해보세요.	비문/불완전

유일한 타당한 대안은 "갑자기 또 힘이드네." (정답 "토닥토닥") → "자신의 감정을 주변 사람들에게 터놓고 이야기해보세요." 1건이었다.

10.5.2 KoBART B2 — 타당한 대안 7건

Q	정답	생성	판정
컴퓨터가 느려졌어	뭘 다운 받으신 건지...	포맷해보세요.	타당한 대안
점점 힘들어져	힘내세요!	조금만 더 버텨요.	타당한 대안
시험 준비 언제 끝나	공부를 많이 해봤나봐요.	합격 기원해요!	타당한 대안
절약해서 부자돼야지	예비 부자님 파이팅!	절약하면 할 수 있어요.	타당한 대안
돈이 엄뎌	퍼가요~	돈 없어도 할 수 있는게 많아요	타당한 대안
세상이 무서워	너무 불안해하지 않아도 돼요.	어흥!! 호랑이보다 무섭나요?	감정 오독
나한테 냄새 날까?	큼큼	깨끗이 씻어보고 섬유유연제나 바디워시,	비문/불완전 (잘림)
다른 사람들은 잘하는것 같아서 초초해	제자리여도 괜찮아요	잘하는 척 알려면 노력해야죠.	비문/불완전

추가로 관찰된 것: KoBART가 "돈이 엄뎌"와 "돈이 안 모여" 두 질문에 **완전히 같은 문장** ("돈 없어도 할 수 있는게 많아요")을 냈다. 비슷한 질문에 같은 답을 재사용하는 경향으로, 학습 데이터가 줄면서 표현의 폭이 좁아진

흔적으로 읽힌다. 다만 공통 평가셋 기준 Distinct-2는 8:1:1과 동일(0.463)했으므로 전체 다양성이 무너진 수준은 아니다.

10.5.3 이 판정이 지표 해석에 미치는 영향

- **KoBART의 BLEU는 과소평가된 값이다.** 표면 불일치 99건(4.2%) 중 70%가 타당한 응답이다. 정답이 하나뿐인 데이터로 표면 유사도를 채점하는 방식의 한계가 8:1:1 실행과 동일하게 재현됐다.
- **표면 불일치 비율이 낮다고 더 나은 모델이 아니다.** 베이스라인의 표면 불일치는 3.9%로 KoBART(4.2%)보다 적지만 내용은 훨씬 나쁘다 — 10건 중 4건이 질문 주제와 무관했다. 자동 규칙만으로 순위를 매기면 안 된다는 점이 두 실행에서 반복 확인됐다.
- **베이스라인의 비문 비율이 8:1:1 때보다 높아졌다**(2건 → 5건). 학습 데이터가 줄면서 문장을 끝까지 만들어내는 능력이 먼저 무너진 것으로 보인다. KoBART는 같은 조건에서 비문이 2건에 그쳤다.

10.6 감정 label별 오류율

model	label	n	반복류	질문복사	표면불일치	평균_ROUGE_L
KoBART B2 lr=5e-5 +warmup	0 일상	1058	1.1	0.8	6.6	26.82
KoBART B2 lr=5e-5 +warmup	1 이별(부정)	700	1.4	0.9	2.7	21.85
KoBART B2 lr=5e-5 +warmup	2 사랑	592	2.7	1.5	1.7	21.54
베이스라인 (최종)	0 일상	1058	0.9	0.0	6.4	23.08
베이스라인 (최종)	1 이별(부정)	700	0.9	0.1	1.6	21.11
베이스라인 (최종)	2 사랑	592	1.9	0.2	2.0	20.4

10.7 대표 실패 사례

오류 유형	모델	Q	정답	생성
빈 응답	베이스라인 (최종)	몸이 아프니까 더 힘드네	더 힘들기 마련이에요.	(빈 응답)
단일 토큰 반복	KoBART B2 lr=5e-5 +warmup	새로운 사랑을 시작하기에 앞서서.	지난 사랑은 걱정하지 말아요.	새로운 시작 시작하기 좋은 계절이에요.
단일 토큰 반복	KoBART B1 lr=2e-5 고정	면세에서 뭐 사지	제 선물 사오세요.	면세점 면세점에서 쇼핑을 해보세요.
구 단위 반복	베이스라인 (최종)	먹고 자고 놀고만 싶어	저도 그러고 싶네요.	서로를 서로를.
구 단위 반복	KoBART (EOS 결함)	사랑할수록 아프다	그 것도 사랑이에요.	사랑하면 아프지 않아요. 자연스러운 현상이에요. 자연스러운 현상이
질문 복사	KoBART B1 lr=2e-5 고정	무언가에 집중하는 남자가 좋아	매력적인 사람이지요.	무언가에 집중해보세요.
질문 복사	KoBART B0 lr=5e-5 고정	걱려가 필요해.	잘하실 거예요!	걱려가 드리고 싶네요.
표면 불일치	KoBART B1 lr=2e-5 고정	입고 나갈 게 없어	아침마다 하는 고민이죠.	파이팅!
표면 불일치	KoBART B1 lr=2e-5 고정	조심조심	안전이 최고죠!	조심하세요.

11. 결론과 향후 과제

11.1 결론

- 전이학습이 베이스라인을 명확히 앞선다. 최고 성능은 KoBART B2 lr=5e-5 +warmup (BLEU 6.46, ROUGE-L 24.01)로, 처음부터 학습한 베이스라인의 최선(베이스라인 (30ep), BLEU 2.94)보다 높다. 학습 비용도 오히려 적다 — 베이스라인 78 epoch vs KoBART 6 epoch.
- 가장 큰 개선은 모델을 바꿔서가 아니라 잘못된 것을 고쳐서 얻었다. 파인튜닝 타킷에 EOS가 누락된 결함을 찾아 고친 것이 단일 변경으로는 가장 큰 효과를 냈다(10.4절 ②).
- loss와 생성 품질은 같지 않다. EOS 결함이 있던 모델도 val_loss는 멀쩡했다. teacher forcing 상태의 loss와 실제 추론(exposure bias)은 조건이 다르므로, 생성문을 직접 채점하는 단계가 반드시 필요하다.
- 지표 하나로로는 판정할 수 없다. 평균 길이 지표가 EOS 결함의 첫 단서였고, corpus 단위 Distinct-n이 아니었다면 "모든 질문에 같은 답"이라는 붕괴는 드러나지 않았을 것이다.
- 구조 선택은 문제 정의에서 나온다. Q→A가 조건부 생성 문제라는 점에서 인코더-디코더가 도출되고, ELECTRA 같은 인코더 전용 모델은 디코더가 없어 적용 자체가 불가능하다.

11.2 이 프로젝트의 한계

- 정답이 하나뿐인 데이터로 채점하고 있다. 10.5절에서 확인했듯 표면 불일치의 상당수가 타당한 응답이며, 이는 현재 BLEU가 전이학습 모델의 성능을 과소평가하고 있다는 뜻이다.
- 각 설정을 시드 하나로 1회씩만 실행했다. 하이퍼파라미터 비교에서 1위(B1, BLEU 7.28)와 2위(B0, 7.10)의 차이는 작아, 시드 변동 범위 안일 가능성을 배제할 수 없다. 결론을 굳히려면 시드를 바꿔 여러 번 실행해 평균과 분산을 함께 봐야 한다.
- label 정보를 학습에 쓰지 않았다. 감정 label을 조건으로 주는 구조는 검증하지 않았다.

11.3 향후 과제

우선순위	할 일	근거
1	사람 평가 또는 의미 기반 지표(BERTScore 등) 추가	10.5절 — 표면 불일치의 상당수가 타당한 응답. 현재 지표가 성능을 과소평가하고 있다
2	베이스라인의 "문맥 무시" 오류 개선	10.5절 — 베이스라인 실패의 주종. 모델 용량·학습량 문제일 가능성
3	시드를 바꿔 하이퍼파라미터 실험 반복	8장 — B1(7.28)과 B0(7.10)의 차이가 작아 시드 변동 범위 안일 가능성을 배제할 수 없다
4	lr을 2e-5보다 더 낮춰(1e-5) 추가 탐색	8장 — lr을 낮춘 B1이 BLEU 1위. 이 방향에 추가 이득이 있는지 확인
5	label 1-2 성능 보강 (가중 샘플링 등)	9.4절 — 데이터 쏠림이 label별 성능 격차로 이어짐
6	label을 조건으로 주는 구조 실험	2.2절 미해결 항목 — (Q,A) 동일·label 상이 행의 활용

11.4 산출물

구분	경로
통합 노트북	notebooks/final_split_712.ipynb (8단계 + 9장 비교, 실행 결과 포함)
지표 · 예측 · 그래프	results/final_712/
모델 체크포인트	checkpoints/final_712/
전처리 데이터	dataset/processed_712/
본 보고서	report/최종결과보고서_7-1-2.pdf

12. 8:1:1 실행과의 비교 — 공통 평가셋에서만

12.1 왜 공통 평가셋에서만 비교하는가

9~11장의 수치는 이 실행의 test **2,350건** 기준이다. 그런데 8:1:1 실행의 수치는 **1,175건** 기준이라 그대로 나란히 놓으면 비교가 성립하지 않는다. 1.3절의 분할 설계 덕분에 이 실행의 test 중 1,175건은 8:1:1의 test와 **완전히 동일한 집합**이고 두 실행 모두 학습에 쓰지 않았다. 그 부분집합에서만 다시 채점해 비교한다.

항목	8:1:1 실행	7:1:2 실행
train	9,400건	8,225건 (-12.5%)
val	1,175건	1,175건 (동일)
이 비교에 쓰는 평가셋	1,175건	같은 1,175건
코드 · 시드 · 하이퍼파라미터 · 디코딩	동일	동일

바뀐 것이 **train 크기 하나뿐**이므로, 이 비교는 "학습 데이터가 12.5% 줄면 무엇이 달라지는가"에 대한 통제 실험이다.

한 가지 남는 차이: 베이스라인의 BPE 토큰라이저와 Word2Vec 임베딩은 각 실행의 train으로 학습되므로 서로 다르다. 이것은 결함이 아니라 "학습 데이터가 줄면 토큰라이저도 그만큼 덜 본다"는 효과의 일부이며, 기존 토큰라이저를 재사용하면 오히려 누수가 된다.

12.2 결과

모델	BLEU(8:1:1)	BLEU(7:1:2)	BLEU Δ	ROUGE-L(8:1:1)	ROUGE-L(7:1:2)	ROUGE-L Δ	Distinct-2(8:1:1)	Distinct-2(7:1:2)
KoBART B1 lr=2e-5 고정	7.28	5.43	-1.85	24.71	23.53	-1.18	0.515	0.473
KoBART B0 lr=5e-5 고정	7.1	5.29	-1.81	24.12	23.65	-0.47	0.482	0.398
KoBART B2 lr=5e-5 +warmup	6.32	6.97	0.65	23.88	24.4	0.52	0.463	0.463
KoBART (EOS 결함)	5.21	3.66	-1.55	19.18	16.48	-2.7	0.384	0.282
베이스라인 (최종)	4.87	2.11	-2.76	22.99	21.96	-1.03	0.303	0.146
베이스라인 (30ep)	1.7	2.84	1.14	22.44	22.57	0.13	0.055	0.043

6개 모델 평균 변화: BLEU **-1.03**, ROUGE-L **-0.79**

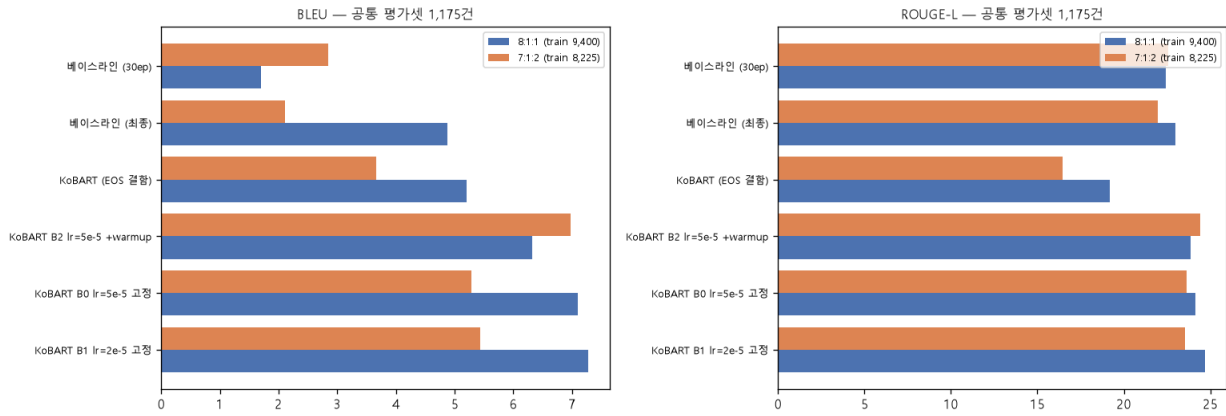


그림 12-1. 분할 비율별 성능 (공통 평가셋 1,175건)

12.3 읽을 수 있는 것

- **학습 데이터 12.5% 감소는 평균적으로 BLEU를 약 1점 떨어뜨린다.** 6개 모델 중 4개가 하락했다. val·평가셋·코드·시드가 전부 같고 train 크기만 다르므로, 이 방향성은 train 크기 변화에 귀속시킬 수 있다.
- **그러나 모델별 편차가 평균보다 크다**(-2.76 ~ +1.14). KoBART B2와 베이스라인 30ep는 오히려 올랐다. 각 설정을 시드 하나로 1회씩만 돌렸으므로 **개별 모델의 변화량을 "데이터가 줄어서 이만큼 나빠졌다"로 읽으면 안 되며, 평균의 방향성만 신뢰할 수 있다.**
- **타격은 베이스라인에 집중됐다.** 처음부터 학습하는 베이스라인이 -2.76으로 가장 크게 떨어진 반면 사전학습 KoBART 계열은 -1.85 ~ +0.65에 머물렀다. **사전학습 지식이 데이터 부족의 충격을 완충한다**는 해석과 일치하며, 이 프로젝트의 중심 주장을 다른 각도에서 뒷받침한다.
- **다양성은 거의 유지됐다.** KoBART B2의 Distinct-2는 0.463 → 0.463으로 변화가 없었다. 데이터가 줄어도 표현의 폭 자체가 무너지지는 않았다.

12.4 주요 결론은 분할 비율을 바꿔도 유지된다

결론	8:1:1	7:1:2	유지 여부
전이학습 > 베이스라인	+49.5%	+187%	유지 (오히려 강화)
EOS 결함 수정이 단일 변경 중 최대 효과	+36.3%	+51.4%	유지
val_loss 순위 ≠ BLEU 순위	3/3 불일치	3/3 불일치	유지
beam5가 greedy보다 우위	+60% / +16%	+49% / +45%	유지
no_repeat_ngram은 사실상 무효	+0.03	-0.01	유지 (부호도 불안정)
최적 하이퍼파라미터 config	B1	B2	바뀜 — 시드 변동 범위 안일 가능성

최적 config가 실행 간에 바뀐 것은 8:1:1 보고서 11.2절에서 한계로 적었던 "각 설정을 시드 하나로 1회씩만 실행했다"는 점이 실제로 드러난 사례다. BLEU 차이가 1점 안팎으로 작아, "lr/스케줄 중 무엇이 최선인가"는 이 두 실행만으로 결론 내릴 수 없고 시드를 바꿔 반복해야 한다.